A process is a program in execution. It consist of several component

Text section: contain program code

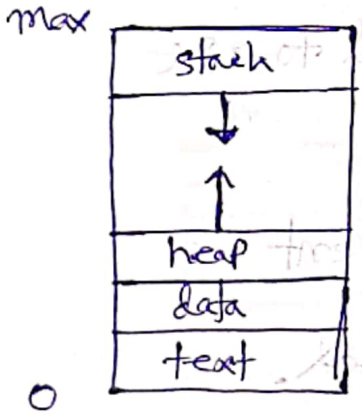
Data section: contain global and static variable

Heap: used for dynamic memory allocation during runtime

Stack: contain local variable, function parameter return address

Program counter: Hold the address of next instruction to be execut

CPU register: store temporary data.



- Representation of a Process in an OS

each Process is represented by Process control block (PCB)

Process ID

Process state (new, Ready, running, waiting, terminated)

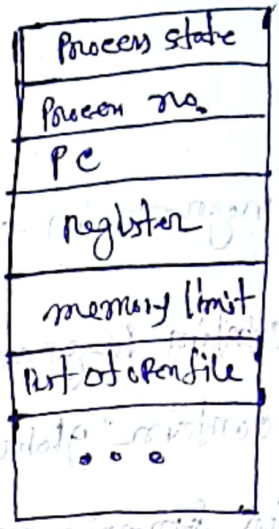
Program counter

CPU register

memory management information

Accounting information:

I/O status information:



Process scheduling is the method used by the OS to decide which process get the CPU.

common Process state:

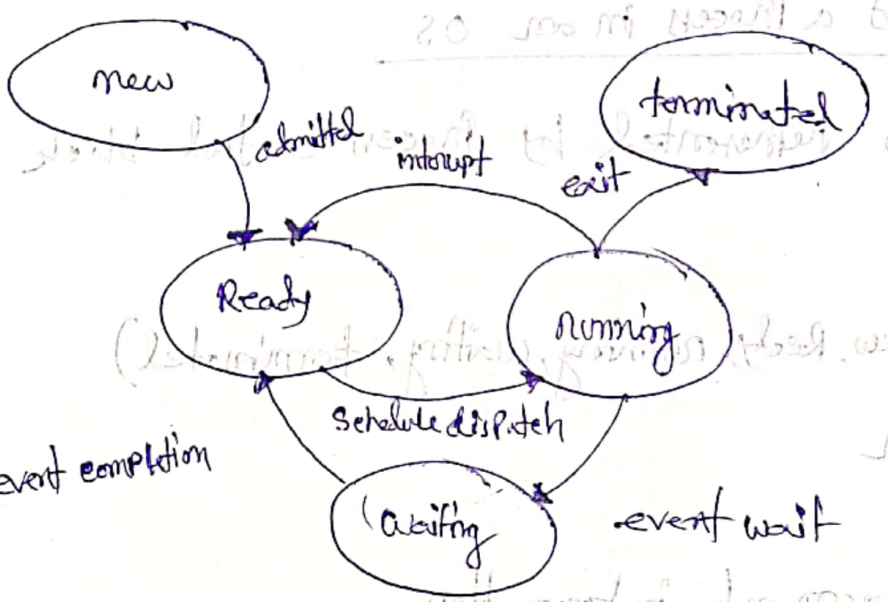
New → Process is being created

Ready → waiting to be assigned to CPU

Running → currently executing

~~Waiting~~ → waiting for an event

Terminated → Execution finished.

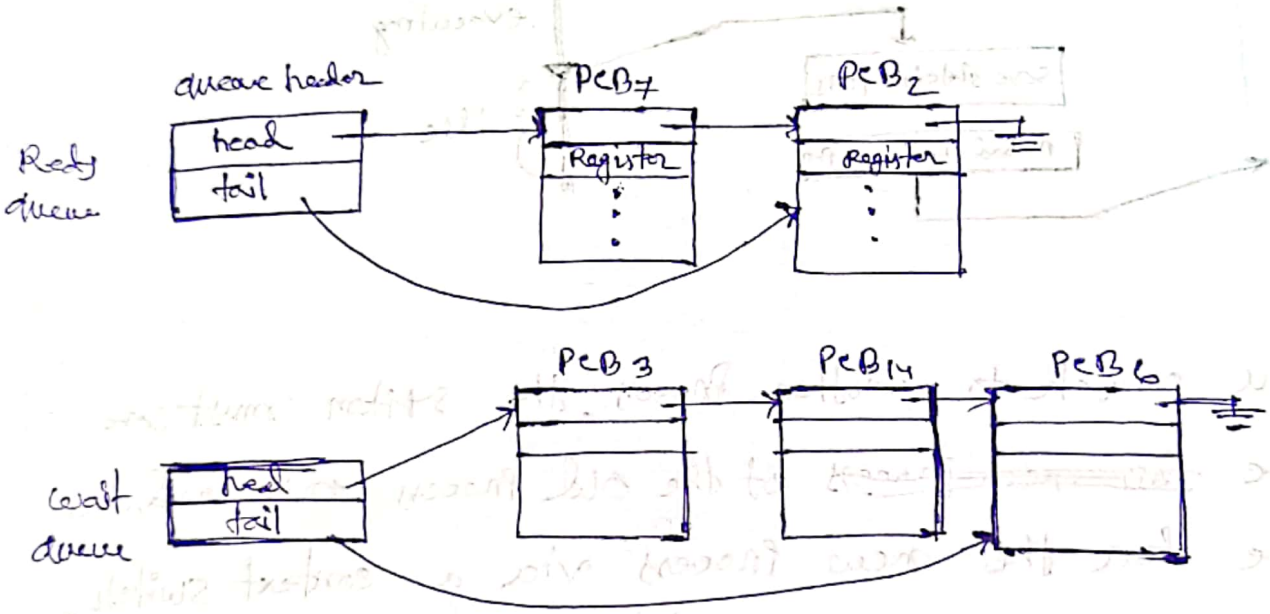


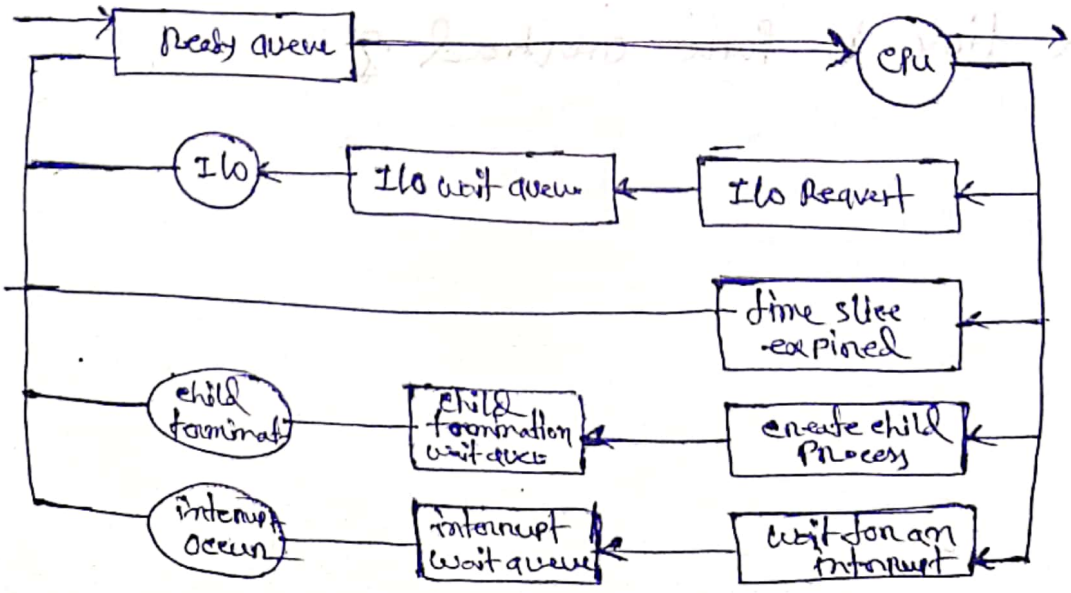
Process scheduler select among available Processes for next execution on CPU core.

Scheduling queue of Process:

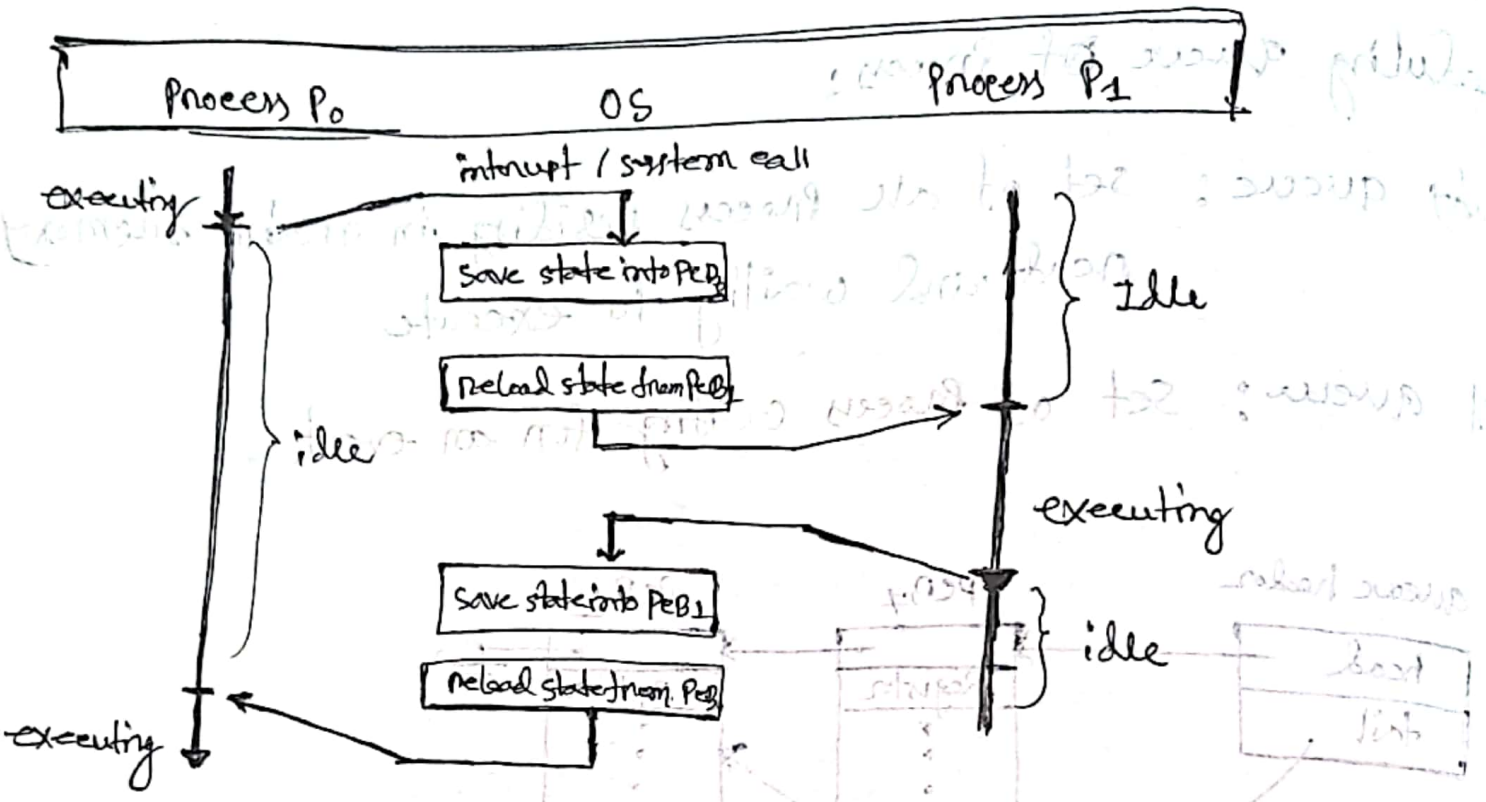
Ready queue: set of all Process residing in main memory, ready and waiting to execute

wait queue: set of Process waiting for an event.





A context switch occurs when the CPU switches from one Process to another.



When CPU switch to another Process, the system must save the state ~~for the new process~~ of the old Process and load the saved state for the new Process via a context switch.

Context of a Process represented in the PCB.

Context switch time is pure overhead.

Process creation : In an OS, a new Process is created when a Parent Process create a child Process

steps: (i) The OS assign a unique Process ID

(ii) A PCB is created.

(iii) memory is allocated (code, data, stack)

(iv) Program load into memory

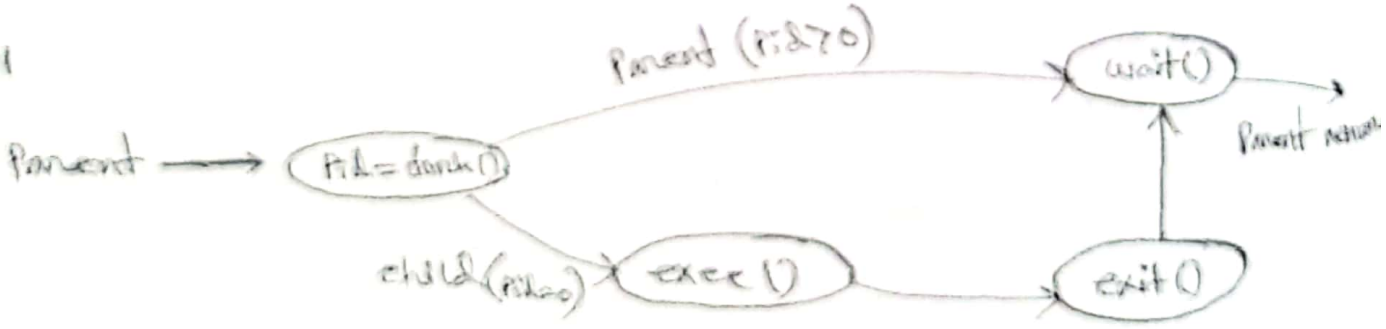
(v) The Process is place in the ready state for execution.

System calls used (UNIX, LINUX)

fork() → create a new child Process

exec() → Load a new Program into the Process

wait() → Parent waits for child Process to finish



Process is terminated when it finished execution

Reason → Normal completion (exit())
crash
Terminate another Process (kill())

System call used:

exit() → Terminates the process itself

wait() → Parent collect child's termination status

kill() → Terminate another Process

If no Parent waiting (did not invoke wait()) Process is a Zomb

If Parent terminate without invoking wait, Process is an Orph

Android process hierarchy:

Foreground Process
visible Process
Service Process
Background Process
Empty Process

1st terminating process that are least important.

- multiprocess Architecture (Chrome Browser) 3 type

Browser process manage user interface, disk and network I/

Render Process render web page, deals with html

Plug in. process for each type of plug in.

- cooperating process need interProcess communication (IPC)

Two model.
Shared memory
message passing

- IPC is a mechanism that allows processes to exchan data and coordinate execution

- Shared memory - multiple processes access a commo memory region.

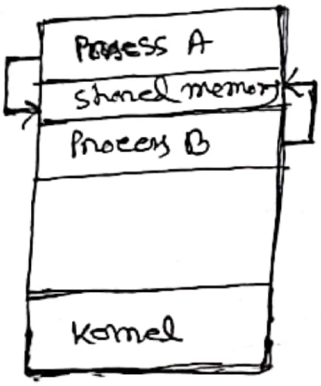
one process write data, other Read it

OS Provide shared memory but Process manage access.

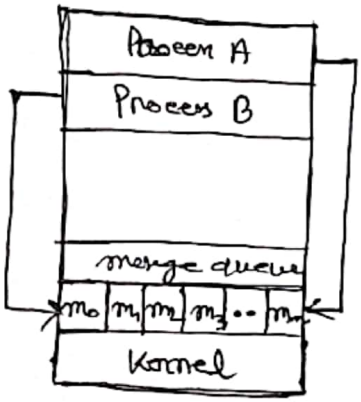
- massge Passing Processes communicate by sending and Receiving messages through the operating system.

no shared memory used

communication is handled via system calls.



Shared memory



massge Passing

shared memory	massge Passing
shared region - used	no shared memory used
Fast speed	slower
synchronization Required	managed by OS
more complex	easier
Data transfer direct access	through OS
less safe	more safe

Implementation of communication link:

Physical → Shared memory
Hardware bus
network

logical → direct / indirect
synchronous / asynch
Automatic

A Pipe is a data channel that unidirectional

Data flow in FIFO

uses Parent and child Process

Design→

create a Pipe using Pipe()

call fork() to create child Process

Parent write to Pipe

child Read from Pipe

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
int main() {
int fd[2];
Pid_t Pid;
char buffer[50];
Pipe(fd);
Pid = fork();
if(Pid>0){
char msg[] = "Hello";
write(fd[1], msg, strlen(msg)+1); }
```

```
else {
    read(fd[0], buffer, sizeof(buffer));
    printf("Received: %s\n", buffer);
}
return 0;
}
```

POSIX Shared memory : Shared memory allow multiple processes to access a common region of memory for communication.

faster IPC mechanism

Process must synchronize access.

Design →

create shared memory using · shm_open()

set size using · ftruncate()

map memory using mmap()

one process write another Read

Write –

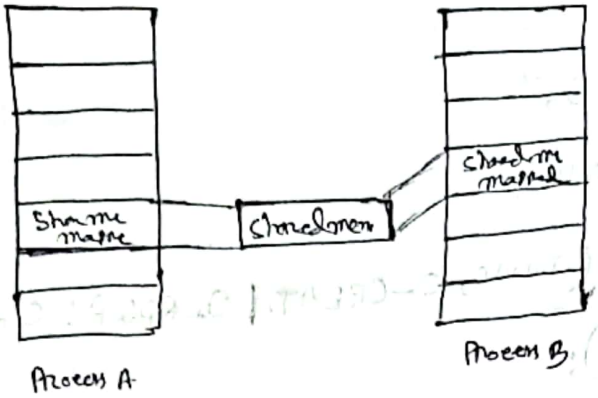
```
#include <stdio.h>
#include <sys/mman.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
```

```
int main() {
  const char *name = "OS";
  const int size = 4096;
  int shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);
  ftruncate(shm_fd, SIZE);
  char *ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);
  sprintf(ptr, "Hello. from. Shared memory");
  return 0;
}
```

Reader

```
#
#
#
#

int main () {
  const
  const
  int. shm_fd = shm_open(name, O_RDONLY, 0666);
  char *ptr = mmap (0, SIZE, PROT_READ, MAP_SHARED, shm_fd
  Printf("Read: %s\n", ptr);
  return ;
}
```



POSIX Shared memory

mach

Ordinary Pipe →

Named Pipe →

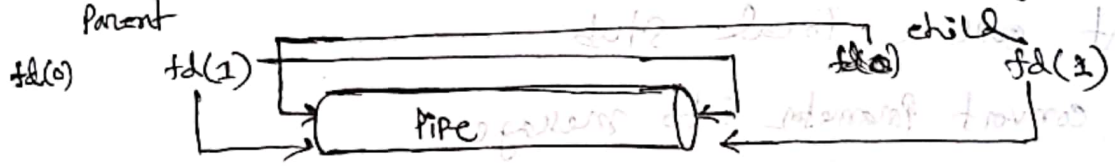
Ordinary Process Pipe are more suitable when communication is needed between related Process, typically a Parent and child Process and communication temporary. unidirectional

Example ⇒ c Program

Parent Process generate numbers

child Process calculate sum

- Require Parent child relationship between communicating Process.



Named Pipe are more suitable when communication is require between two independent Processes. Persistent, bidirectional.

A log monitoring system.

Program A → write log

Program B → reads logs and analyzes.

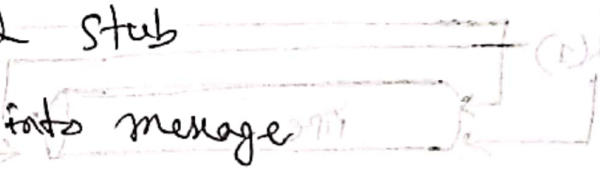
A Socket is an endPoint for communication between two Processes over a network

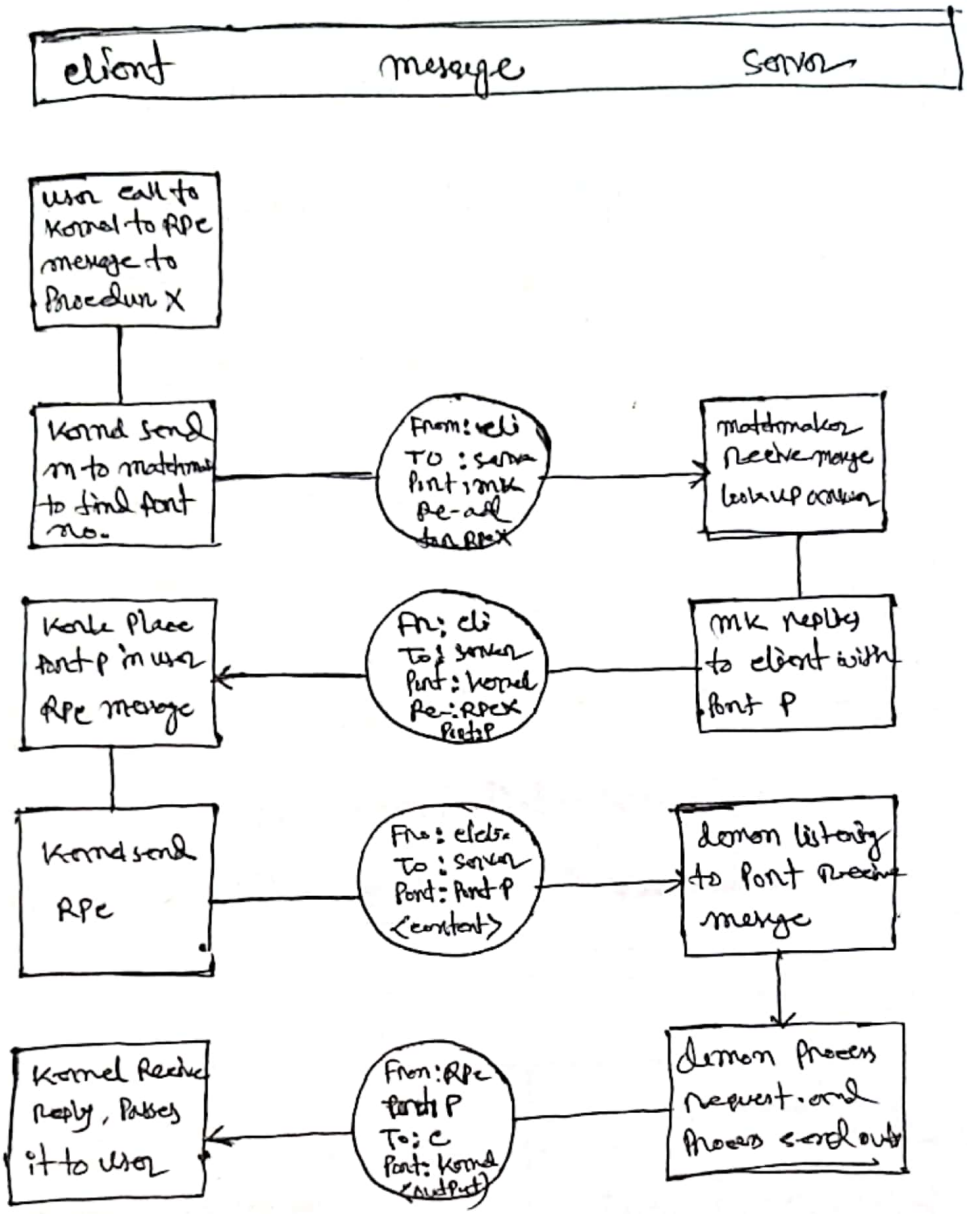
How work:

Server create a socket and binds it to a Port
server listen for incoming connection
client create socket and connect to server
Data exchange using, Send(); Recv()

Remote Procedure call (Rpc) allow a Program to call a function on a remote machine as if it were a local function

work

client call a local stub
stub convert Parameter into message

message sent to server
Server stub receive and execute Procedure
Result send back to client


client
message
Server
User call to Kernel to RPC message to Procedur X
Kernel send m to matchmaker to find Port no.
From: cli TO : server Port : mk Re-call
matchmaker Receive msge lookup
mk reply to client with Port P
Kernel Place Port P in user RPC msge
From: cli To : server Port : Kernel Re: RPCX Port P
Kernel send RPC
From: client To : server Port: Port P <content>
demon listening to Port Receive msge
Kernel Receive reply, Passes it to user
From: RPC Port: P To: C Port: Kernel <output>
demon Process request. and Process <output>